Project	VLSI Strength	Signals/Systems Fit	Originality	Implementation Risk	Best Story
DCT/IDCT Engine	Very high	High	Medium-low	Medium	Classic image/video DSP accelerator with algorithm/architecture co-design
Viterbi Decoder	Very high	Medium	Medium-low	Medium-high	Communication decoder with ACS pipelining, survivor memory, and throughput/area tradeoffs

1. DCT/IDCT Engine Compared to Lock-In DSP

The **DCT/IDCT engine** is probably the most textbook-clean Advanced VLSI project.

It naturally gives you:

```
2D transform architecture
row-column decomposition
transpose buffer
butterfly datapath
constant multipliers
pipeline stages
fixed-point quantization
throughput/area tradeoff
```

The main hardware flow is:

```
8×8 input block
   ↓
row-wise 1D DCT
   ↓
transpose buffer
   ↓
column-wise 1D DCT
   ↓
quantized DCT coefficients
```

This is very strong because it has a clean algorithm-to-hardware mapping.

The downside is that **DCT engines have been done many, many times**. JPEG/MPEG DCT accelerators are standard VLSI projects. So if you choose this, the project cannot just be:

> "I implemented a 2D DCT in Verilog."

That would look generic.

To make it graduate-level, the story should be:

> **We compare DCT algorithms and microarchitectures under area/timing/precision constraints.**

For example:

```
Direct matrix DCT
vs. Loeffler DCT
vs. AAN scaled DCT

and
```

```
unpipelined
vs. pipelined
vs. folded/resource-shared
```

That gives a strong VLSI design-space exploration.

Compared to the lock-in idea:

Aspect	DCT/IDCT	Lock-In Resonance Tracker
More standard VLSI topic?	Yes	Less standard
More original?	No	Yes
Easier to explain to professor?	Yes	Medium
Better for image/video DSP?	Yes	No
Better for weak-signal/sensor/RF theme?	No	Yes
Strong pipelining story?	Yes	Yes
Strong fixed-point story?	Yes	Yes

So DCT is **safer and more canonical**, but less original.

---

2. Viterbi Decoder Compared to Lock-In DSP

The **Viterbi decoder** is also a very strong VLSI project, but it is more communication-systems oriented than signal-filtering oriented.

Its architecture naturally includes:

```
branch metric unit
path metric update
ACS units
survivor memory
traceback unit
control FSM
```

The core bottleneck is the ACS operation:

```
Add → Compare → Select
```

This gives a very clean critical-path discussion.

A typical flow is:

```
received soft bits
   ↓
branch metric computation
   ↓
ACS trellis update
   ↓
survivor path storage
   ↓
traceback
   ↓
decoded bits
```

This is excellent for VLSI because you can study:

```
radix-2 vs radix-4 trellis
parallel ACS units
traceback depth
register-exchange vs traceback memory
soft-decision bit width
path metric normalization
```

However, I would be careful with the phrase:

> "Replace expensive comparisons with approximations."

Comparators are not usually the biggest cost compared to ACS parallelism, memory, and traceback logic. A stronger framing would be:

```
branch metric quantization
path metric bit-width reduction
survivor memory optimization
radix-4 ACS restructuring
traceback-depth vs BER tradeoff
```

Compared to the lock-in project:

Aspect	Viterbi Decoder	Lock-In Resonance Tracker
More classic VLSI architecture?	Yes	Less classic
Stronger control/memory component?	Yes	Medium
Stronger signal-filtering component?	No	Yes
More algorithmic complexity?	Yes	Medium
More implementation/debug risk?	Higher	Medium
More original application angle?	Lower	Higher

The Viterbi decoder is probably the **most architecture-heavy** of the three.

---

3. Which One Is Best?

It depends on what you want the project to look like.

If you want the safest Advanced VLSI project

Choose:

DCT/IDCT Engine

It has the cleanest course alignment:

```
pipelining
retiming
butterfly datapath
constant multipliers
fixed-point arithmetic
transpose memory
throughput/area tradeoff
```

A strong title would be:

Algorithm-Architecture Co-Design of a Pipelined 2D-DCT Engine for JPEG-Style Compression

This is safe, but not very original unless you compare multiple architectures.

If you want the most architecture-rich project

Choose:

Viterbi Decoder

It has excellent VLSI depth:

```
ACS critical path
survivor memory
traceback architecture
parallel trellis processing
soft-decision precision
radix-2 vs radix-4 tradeoff
```

A strong title would be:

Throughput-Optimized VLSI Architecture for a Soft-Decision Viterbi Decoder

This is very graduate-level, but the debugging and verification burden may be higher.

If you want the most original and research-aligned project

Choose:

Lock-In Resonance-Tracking DSP Accelerator

This is less common than DCT/Viterbi and connects better to sensing, RF, resonators, and your broader research background.

Its strength is originality:

```
weak-signal detection
I/Q demodulation
FIR/IIR filtering
energy accumulation
resonance tracking
critical-path-aware DSP pipeline
```

A strong title would be:

Critical-Path-Aware Pipelined Lock-In DSP Accelerator for Real-Time Resonance Tracking

This is less textbook, but more unique.

My Ranking for You

For your case, I would rank them like this:

1. DCT/IDCT Engine — best course fit

This is the most professor-friendly Advanced VLSI project.

2. Lock-In Resonance Tracker — most original

This is the best if you want the project to stand out and connect to RF/sensing/quantum-style instrumentation.

3. Viterbi Decoder — strongest architecture depth

This is excellent, but slightly less aligned with your stated FIR/IIR/signals-systems direction unless the course also emphasizes communication systems.

My Recommendation

If the professor expects a **classic VLSI accelerator**, choose the **DCT/IDCT engine**.

If you want something that feels more like **your own project**, choose the **lock-in resonance tracker**.

The strongest compromise would be:

> **Do the DCT/IDCT engine, but make the contribution a design-space exploration: direct vs Loeffler/AAN, pipelined vs folded, and 12-bit vs 16-bit fixed-point.**

That will look graduate-level, clean, measurable, and defensible.

For this: 'Lock-In Resonance-Tracking DSP Accelerator' do we have to do everything from scratch? what tools will we need? vivado and all?

Thought for 4s

No — you **do not have to do everything from scratch**.

For **"Lock-In Resonance-Tracking DSP Accelerator,"** the original work should be your **RTL architecture and VLSI analysis**, not manually deriving every sine table, filter coefficient, or test signal from scratch.

You should build the **main Verilog/SystemVerilog datapath yourself**, but use Python/MATLAB for reference models, coefficient generation, test vectors, and plotting.

What You Should Build Yourself

These should be your own RTL modules:

```
1. Phase accumulator / NCO controller
2. sine/cosine LUT interface
3. I/Q mixer
4. FIR low-pass filter
5. optional IIR smoothing filter
6. magnitude / energy estimator
7. frequency-bin accumulator
8. peak/dip resonance detector
9. top-level pipeline
10. testbench
```

The important "from scratch" part is:

> how you organize the datapath, pipeline it, manage fixed-point widths, and compare timing/area/accuracy.

You do **not** need to manually invent filter coefficients or sine values.

What You Can Generate Using Python/MATLAB

You can use Python or MATLAB to generate:

```
sine/cosine lookup table
FIR coefficients
IIR coefficients
noisy input samples
expected golden-model outputs
plots for the report
```

For example, Python can generate:

```
sin_rom.mem
cos_rom.mem
input_samples.mem
expected_energy.csv
```

Then your Verilog testbench reads those files.

That is completely normal and not "copying."

Tools You Need

Minimum Tool Set

For a complete project, you need:

```
Verilog/SystemVerilog simulator
synthesis/timing tool
Python or MATLAB
waveform viewer
```

A practical setup would be:

Purpose	Tool
RTL design	Verilog/SystemVerilog
Simulation	Vivado Simulator, ModelSim/Questa, Icarus Verilog, or Verilator
Synthesis/timing	Vivado, Quartus, or Yosys
Waveform viewing	GTKWave or Vivado waveform viewer
Signal generation/plots	Python: NumPy, SciPy, Matplotlib
Report	LaTeX/Word/PowerPoint

Do You Need Vivado?

Not strictly, but **Vivado is a very good choice** if your course expects FPGA-style VLSI results.

Use Vivado if you want:

```
synthesis report
timing report
critical path
LUT/FF/DSP usage
Fmax estimate
FPGA resource utilization
```

You do **not** need an FPGA board unless the professor requires hardware deployment.

You can do:

```
RTL simulation → synthesis → timing/resource report
```

without programming a board.

Best Tool Flow for This Project

I would recommend this flow:

```
Python golden model
       ↓
Generate test vectors and LUT files
       ↓
Verilog/SystemVerilog RTL
       ↓
RTL simulation
       ↓
Waveform/debug
       ↓
Vivado synthesis
       ↓
Timing/resource comparison
       ↓
Python plots + final report
```

This is clean and defensible.

Suggested Tool Choice

Best practical choice

Use:

```
Python + SystemVerilog + Vivado
```

because Vivado gives you the reports you need for a VLSI course.

You can synthesize for a generic Xilinx FPGA part, for example an Artix-7 or Zynq device, even without using a board.

What Vivado Gives You

Vivado can produce the key result tables:

Result	Why it matters
Critical path delay	Shows bottleneck

Result	Why it matters
Maximum clock frequency	Shows speed
LUT usage	Approximate logic area
FF usage	Register/pipeline overhead
DSP usage	Multiplier cost
Timing report	Shows whether pipelining helped

That is exactly what you need for the project.

What Python Gives You

Python is useful for the signal-processing side:

```
generate noisy input signal
generate resonance sweep
generate sine/cosine LUT
generate FIR coefficients
compare RTL output with golden model
plot energy vs frequency bin
plot detection accuracy vs noise
plot fixed-point error
```

So Python supports the project, but the main design is still Verilog.

What You Should Avoid

Avoid using ready-made DSP IP cores for the main blocks.

For example, do **not** just instantiate:

```
Xilinx FIR Compiler IP
Xilinx DDS Compiler IP
Xilinx CORDIC IP
```

because then the project becomes mostly IP integration.

You can mention those as industrial alternatives, but your actual design should implement the important blocks yourself.

Using inferred multipliers is fine:

```
assign y = a * b;
```

Vivado may map them to DSP slices automatically. That is acceptable.

Recommended Project Scope

For the full version, implement:

```
1. Python golden model
2. Verilog NCO with sine/cos LUT
```

```
3. I/Q mixer
4. FIR low-pass filter
5. magnitude-squared estimator
6. bin accumulator
7. resonance detector
8. baseline architecture
9. pipelined architecture
10. Vivado synthesis comparison
```

Optional advanced additions:

```
IIR smoothing
folded/resource-shared multiplier version
8/12/16-bit precision sweep
FIR tap-count sweep
peak vs dip detection mode
```

Best Final Tool Stack

Use this:

```
Python        → test vector/LUT/coefficient generation + plots
SystemVerilog → RTL design and testbench
Vivado        → simulation, synthesis, timing, resource report
GTKWave       → optional waveform viewing
```

So the answer is:

> You do not need to do everything from scratch, but the RTL datapath and architecture comparison should be your own. Vivado is not absolutely required, but it is the best choice if you want strong synthesis, timing, critical-path, and resource-utilization results for a graduate VLSI project.

Yes we will use vivado. So break it down for me. What is the goal, what are the target's/checks, and what are the phases? What is the plan for each phase with respect to implementation and target?

Thought for 7s

Yes. With Vivado, the project should be structured as a **hardware accelerator design + VLSI analysis project**, not just a simulation project.

The full project goal is:

> **Design, implement, and evaluate a fixed-point Verilog/SystemVerilog lock-in DSP accelerator that tracks a resonance peak/dip from a swept noisy signal stream, then compare baseline and optimized microarchitectures using Vivado timing/resource reports.**

The main result should be:

> **The accelerator correctly detects the resonance bin, and the pipelined architecture improves timing/Fmax by reducing critical-path bottlenecks at the cost of latency/register overhead.**

---

1. Overall System Goal

The accelerator receives a sampled signal stream:

```
sample_in[n]
```

The signal contains a weak sinusoidal/resonance response buried in noise. The hardware performs lock-in-style DSP:

```
Input sample
   ↓
NCO reference generation
   ↓
I/Q mixing
   ↓
FIR low-pass filtering
   ↓
Magnitude-squared estimation
   ↓
Per-frequency-bin accumulation
   ↓
Peak/dip resonance detection
   ↓
Detected resonance bin
```

The final output is:

```
detected_bin
best_energy
done
```

For example:

```
True resonance bin: 5
Hardware detected bin: 5
PASS
```

---

2. Main Project Targets

Functional Targets

The design should prove that the accelerator works correctly.

Target	Check
NCO generates correct sine/cosine references	Compare LUT/reference waveform against Python
I/Q mixer produces expected mixed signal	Check RTL output vs Python golden model
FIR removes high-frequency/noise components	Compare filtered RTL output vs expected output
Magnitude estimator computes signal strength	Check `I² + Q²` behavior
Bin accumulator stores energy per frequency bin	Print/plot energy per bin
Resonance tracker finds correct bin	`detected_bin == true_bin`
Peak/dip mode works	Peak mode detects max; dip mode detects min

Minimum functional success:

```
For a known synthetic resonance sweep, the RTL detects the correct resonance bin.
```

VLSI/Architecture Targets

This is what makes it graduate-level.

Target	Check in Vivado
Identify baseline critical path	Timing report
Improve timing using pipelining	Compare Fmax baseline vs pipelined
Measure register overhead	FF count
Measure logic/multiplier cost	LUT/DSP count
Compare latency	Cycle count from input to output
Compare throughput	Samples per cycle
Study fixed-point cost	12-bit vs 16-bit vs 24-bit optional
Discuss bottleneck shift	Timing path before/after pipelining

Expected architectural result:

```
Pipelined design should have higher Fmax than baseline.
```

A good target is:

```
Pipelined Fmax ≈ 1.5× to 3× baseline Fmax
```

The exact number depends on FPGA part, bit width, and how Vivado maps multipliers.

---

3. Recommended Vivado Target

You do not need an FPGA board unless required.

Use a common Xilinx FPGA target, for example:

```
Artix-7: xc7a35tcpg236-1
```

or whatever FPGA part your course uses.

Use Vivado for:

```
RTL simulation
synthesis
timing analysis
resource utilization
critical path report
```

Do not rely on Xilinx FIR/DDS/CORDIC IP cores for the main design. The main RTL should be yours.

---

4. Phase-by-Phase Plan

Phase 0 — Freeze Specification

Goal

Define the system parameters before implementation.

Implementation Decisions

Use a parameterized design, but start with fixed values:

```
Input width: 16-bit signed
Reference width: 16-bit signed
Mixer output: 32-bit signed
FIR taps: 8 or 16
Number of bins: 8 or 16
Samples per bin: 64 or 128
Magnitude width: 64-bit unsigned
Accumulator width: 64-bit or wider
```

Recommended starting point:

```
16-bit samples
8 bins
64 samples/bin
8-tap FIR
peak-detection mode
```

Target/Check

At the end of this phase, you should have:

```
A fixed block diagram
A fixed signal format
A list of RTL modules
A Vivado target part
```

Phase 1 — Python Golden Model and Test Vectors

Goal

Create the signal-processing reference model before writing RTL.

What Python Should Generate

Python can generate:

```
sin_rom.mem
cos_rom.mem
input_samples.mem
expected_energy_per_bin.csv
expected_detected_bin.txt
```

The synthetic input should model a frequency sweep.

Example:

```
bin 0: low response
bin 1: medium response
bin 2: high response
bin 3: resonance peak
bin 4: high response
```

```
bin 5: medium response
bin 6: low response
bin 7: low response
```

You can add controlled noise.

Target/Check

Python should produce:

```
Expected resonance bin
Expected energy per bin
Reference plots
```

Minimum success:

```
Python model detects the correct resonance bin before RTL implementation begins.
```

This becomes your golden reference.

Phase 2 — NCO / Reference Generator RTL

Goal

Generate sine and cosine reference signals in hardware.

Implementation

Use:

```
phase accumulator + sine/cosine ROM
```

The phase accumulator updates every sample.

```
phase <= phase + phase_step
```

The upper phase bits index into sine/cosine lookup tables.

The LUT values are loaded from `.mem` files generated by Python.

Target/Check

Check that RTL outputs:

```
sin_ref
cos_ref
```

match the expected LUT sequence.

Simulation check:

```
sin_ref and cos_ref repeat periodically
cos_ref is phase-shifted from sin_ref
values match Python-generated table
```

Vivado check:

```
ROM inferred correctly
No timing issue yet
```

Phase 3 — I/Q Mixer RTL

Goal

Mix the input signal with sine and cosine references.

Implementation

Compute:

```
I_mixed = sample_in × cos_ref
Q_mixed = sample_in × sin_ref
```

This produces the lock-in I/Q channels.

Target/Check

Simulation check:

```
I_mixed and Q_mixed match Python fixed-point model within rounding tolerance
```

Vivado check:

```
DSP usage appears in utilization report
Multiplier path appears in timing report
```

Expected resource:

```
2 DSP blocks for I/Q multiplication
```

depending on bit width and FPGA mapping.

Phase 4 — FIR Low-Pass Filter RTL

Goal

Filter the mixed I/Q signals to remove high-frequency components.

Implementation

Use the same FIR module twice:

```
FIR_I for I_mixed
FIR_Q for Q_mixed
```

Start with symmetric low-pass coefficients, for example:

```
[1, 2, 3, 4, 4, 3, 2, 1]
```

Normalize using a right shift.

Baseline FIR

Direct or mostly direct MAC structure.

Expected bottleneck:

```
coefficient multiply/shift-add → adder chain → output
```

Target/Check

Simulation check:

```
FIR output follows Python golden output
filtered output is smoother than mixed input
```

Vivado check:

```
FIR MAC/adder tree likely becomes a critical path
```

Minimum success:

```
Both I and Q filtered outputs are valid and numerically reasonable.
```

Phase 5 — Magnitude-Squared Estimator

Goal

Estimate signal strength without square root.

Implementation

Compute:

```
mag_sq = I_filtered² + Q_filtered²
```

Do not implement square root. For peak detection, squared magnitude is enough.

Target/Check

Simulation check:

```
mag_sq increases near the resonance bin
mag_sq remains nonnegative
no overflow under chosen bit width
```

Vivado check:

```
Squaring may use DSP blocks
Magnitude path may become a new timing bottleneck
```

Expected bottleneck candidate:

```
square + add path
```

Phase 6 — Bin Accumulator RTL

Goal

Accumulate signal energy for each frequency bin.

Implementation

For each bin:

```
bin_energy = bin_energy + mag_sq
```

After `SAMPLES_PER_BIN` samples:

```
current bin complete
send bin_energy to resonance tracker
reset accumulator
move to next bin
```

Target/Check

Simulation output should show something like:

```
Bin 0 energy = ...
Bin 1 energy = ...
Bin 2 energy = ...
Bin 3 energy = maximum
...
```

Minimum success:

```
Energy per bin follows the expected resonance profile.
```

Phase 7 — Peak/Dip Resonance Tracker

Goal

Detect the resonance bin.

Implementation

Peak mode:

```
if bin_energy > best_energy:
    best_energy = bin_energy
    detected_bin = current_bin
```

Optional dip mode:

```
if bin_energy < best_energy:
    best_energy = bin_energy
    detected_bin = current_bin
```

At the end of the full sweep:

```
done = 1
```

Target/Check

Simulation check:

```
detected_bin == expected_bin
done asserted after final bin
best_energy matches largest/smallest bin energy
```

Minimum success:

```
Correct detection for clean input and noisy input.
```

Phase 8 — Baseline Top-Level Integration

Goal

Connect all modules into one baseline accelerator.

Baseline Pipeline

```
sample_in
   ↓
NCO
   ↓
I/Q mixer
   ↓
baseline FIR
   ↓
magnitude-squared
   ↓
accumulator
   ↓
tracker
```

Target/Check

Functional check:

```
Full RTL system detects correct bin.
```

Vivado synthesis check:

```
Design synthesizes successfully
Timing report identifies critical path
Utilization report gives LUT/FF/DSP counts
```

Baseline report items:

```
Critical path delay
Estimated Fmax
LUT count
FF count
DSP count
Latency
Throughput
```

This is your reference design.

Phase 9 — Pipelined Architecture

Goal

Reduce the critical path using pipeline registers.

Where to Pipeline

Pipeline around the heavy datapath sections:

```
Stage 1: input sample + NCO reference register
Stage 2: I/Q multiplication
Stage 3: FIR products
Stage 4: FIR partial sums
Stage 5: FIR final sum
Stage 6: magnitude square
Stage 7: accumulation
Stage 8: peak/dip comparison
```

The biggest target is the FIR adder tree and magnitude path.

Important Design Point

Pipelining changes latency, so valid signals must be delayed with the data.

You need to pipeline:

```
sample_valid
bin_id
end_of_bin
end_of_sweep
```

alongside the datapath.

Target/Check

Functional check:

```
Pipelined design still detects the same bin as baseline.
```

Timing check:

```
Pipelined Fmax > baseline Fmax
```

Resource check:

```
FF count increases
LUT/DSP count should be similar or slightly higher
```

Expected result:

```
Critical path moves from FIR MAC chain to another shorter block.
```

This is the central VLSI result.

Phase 10 — Optional Folded / Resource-Shared Architecture

Goal

Show area-throughput tradeoff.

Implementation Idea

Instead of using fully parallel I/Q FIRs, reuse fewer multipliers over multiple cycles.

For example:

```
One multiplier reused across FIR taps
or
one FIR datapath reused for I and Q on alternating cycles
```

Target/Check

Expected:

```
Lower DSP usage
Lower throughput
More control logic
Higher latency
```

Vivado check:

```
DSP count decreases
FF/LUT for controller may increase
Throughput drops
```

This phase is optional, but it gives a strong graduate-level design-space exploration.

Phase 11 — Optional IIR Smoothing Stage

Goal

Compare FIR-only and FIR+IIR smoothing.

Implementation

Simple first-order IIR:

```
y[n] = αx[n] + (1 - α)y[n-1]
```

Use shift-friendly coefficients, for example:

```
α = 1/4
```

so multiplication can be implemented with shifts.

Target/Check

Signal check:

```
IIR smoothing reduces energy noise
but increases response latency
```

VLSI check:

```
IIR feedback path becomes difficult to pipeline
```

This gives a useful FIR-vs-IIR discussion.

Phase 12 — Fixed-Point Precision Sweep

Goal

Analyze accuracy vs hardware cost.

Variants

Run the same design with:

```
12-bit input
16-bit input
24-bit internal path
```

Target/Check

For each precision, collect:

```
detected bin
energy error vs Python
Fmax
LUT/FF/DSP usage
overflow behavior
```

Expected:

```
Lower precision → less area, possibly higher Fmax, but worse detection under noise.
Higher precision → better accuracy, more area, potentially lower Fmax.
```

This is an excellent report section.

Phase 13 — Vivado Synthesis and Timing Analysis

Goal

Collect the main VLSI evidence.

For Each Architecture

Run synthesis and collect:

```
Timing summary
Worst negative slack
Critical path delay
Estimated Fmax
LUT usage
FF usage
DSP usage
BRAM usage
```

Use a clock constraint such as:

```
100 MHz for baseline initial check
200 MHz or higher for pipelined stress test
```

Then compare whether each design meets timing.

Target/Check

Main comparison table:

Design	Fmax	Critical Path	LUTs	FFs	DSPs	Latency
Baseline	lower	FIR/magnitude	medium	lower	high	low
Pipelined	higher	shorter path	medium	higher	high	higher
Folded	medium	controller/multiplier	lower	medium	lower	high

Design	Fmax	Critical Path	LUTs	FFs	DSPs	Latency
FIR+IIR	medium	IIR feedback	higher	higher	high	higher

This table is the core project result.

Phase 14 — Report and Presentation

Goal

Turn the implementation into a graduate-level VLSI story.

Report Structure

Use this structure:

```
1. Motivation
2. Signal model
3. Lock-in resonance-tracking algorithm
4. Fixed-point hardware architecture
5. Baseline RTL design
6. Pipelined RTL design
7. Optional folded/IIR/precision variants
8. Vivado simulation and synthesis setup
9. Functional correctness results
10. Timing/resource/critical-path analysis
11. Discussion
12. Conclusion
```

Key Message

The main conclusion should be:

> The baseline lock-in DSP accelerator correctly tracks resonance, but its timing is limited by multiplier-heavy and adder-tree datapaths. Pipelining the I/Q, FIR, and magnitude stages reduces the critical path and improves Fmax while preserving detection correctness, at the cost of increased latency and register usage. Fixed-point precision and filter structure create additional tradeoffs between hardware cost and signal-detection accuracy.

Final Phase Summary Table

Phase	Implementation	Target/Check
0	Freeze parameters	Fixed spec and FPGA target
1	Python golden model	Expected bin and energy profile
2	NCO RTL	Correct sine/cos sequence
3	I/Q mixer	RTL matches fixed-point model
4	FIR RTL	Filtered I/Q outputs valid
5	Magnitude estimator	Correct $I^2 + Q^2$
6	Bin accumulator	Energy per bin computed

Phase	Implementation	Target/Check
7	Resonance tracker	Correct detected bin
8	Baseline top	Full system works
9	Pipelined top	Same output, better timing
10	Folded design optional	Lower DSP usage
11	IIR optional	Smoothing vs feedback bottleneck
12	Precision sweep optional	Accuracy vs area/timing
13	Vivado synthesis	Fmax/resource/critical path
14	Report	VLSI tradeoff story

Minimum Required Completion

To make the project complete, you need at least:

```
Python golden model
Verilog/SystemVerilog baseline accelerator
Verilog/SystemVerilog pipelined accelerator
RTL simulation showing correct resonance detection
Vivado synthesis report for both
Timing/resource comparison
Critical-path discussion
```

Strong Graduate-Level Completion

For a stronger version, add:

```
fixed-point precision sweep
FIR tap-count sweep
optional IIR smoothing
optional folded/resource-shared datapath
peak vs dip detection mode
```

The main thing is this:

> The project should not be judged only by whether the DSP works. It should be judged by whether you can explain how the architecture changes the critical path, Fmax, latency, area, and detection reliability.

generate a markdown file for the project plan as given here